Python for Data Analytics 2

Session 1: Dates and Times with Python and Pandas

Session Objectives

- Understand why dates/times matter in analytics
- Learn Python's built-in date/time tools
- Master pandas date/time types and conversions
- Generate and manipulate date ranges for business
- Resample, shift, and roll time series data
- Practice with real-world business examples

Why Dates & Times Matter in Analytics

- **Sales trends:** How do sales change over time?
- **Customer behavior:** When do users interact with your product?
- **Forecasting:** Predicting future values
- **Operations:** Scheduling, deadlines, reporting

Python's Built-in Date & Time Tools

- `datetime` module: native date/time objects
- `dateutil`: powerful date string parsing

```python
from datetime import datetime
from dateutil import parser

# Create a datetime for a transaction
transaction_time = datetime(2025, 7, 20, 15, 45)
# datetime.datetime(2025, 7, 20, 15, 45)

# Parse business date strings given some text
order_date = parser.parse("July 20, 2025")
# datetime.datetime(2025, 7, 20, 0, 0)
```

Pandas: Timestamps

- Represent a single point in time (like Python's `datetime`)
- Used for precise events: transactions, log entries, etc.

```
import pandas as pd
# Example: Transaction timestamp
transaction_time = pd.Timestamp('2025-07-20 10:00')
# 2025-07-20 10:00:00

print(type(transaction_time))
# <class 'pandas._libs.tslibs.timestamps.Timestamp'>
```

Pandas: Periods

- Represent a span of time (e.g., month, quarter, year)
- Useful for business cycles, reporting periods, etc.

```
# Example: Q3 2025 as a period
q3_2025 = pd.Period('2025Q3')
print(q3_2025)
print(q3_2025.start_time, q3_2025.end_time)
# 2025Q3
# 2025-07-01 00:00:00 2025-09-30 23:59:59.999999999
```

Pandas: Timedeltas

- Represent a duration or difference between two dates/times
- Useful for calculating time intervals, delays, durations

```
# Example: Meeting duration of 2.5 hours
meeting_duration = pd.to_timedelta(2.5, unit="H")
# 0 days 02:30:00

# Add duration to a timestamp
end_time = pd.Timestamp('2025-07-20 10:00') + meeting_duration
# 2025-07-20 12:30:00
```

Converting Strings to Dates in Pandas

- Use `pd.to_datetime()` for flexible conversion
- Handles mixed formats and business data

```
dates = pd.Series(["20/07/2025", "July 31, 2025", "2025-07-04"])
pd.to_datetime(dates)
# 0   2025-07-20
# 1   2025-07-31
# 2   2025-07-04
# dtype: datetime64[ns]
```

Indexing & Slicing Time Series Data

- Set date columns as index for time-based analysis
- Slice by year, month, or custom ranges

```
index = pd.DatetimeIndex([
    '2025-07-01', '2025-07-08', '2024-07-15', '2024-08-22'])

data = pd.Series([100, 200, 150, 300], index=index)
data['2024-07']  # Filter July 2024
# 2024-07-15    150
# Freq: W-SUN, dtype: int64

data['2025']     # Filter all 2025
# 2025-07-01    100
# 2025-07-08    200
# Freq: W-SUN, dtype: int64
```

Generating Date Ranges: pd.date_range()

- `pd.date_range()` : regular timestamp sequences (e.g., daily, hourly)

```
# Daily range for July 2025
dates = pd.date_range('2025-07-01', '2025-07-03', freq="D")
print(dates)
# DatetimeIndex(['2025-07-01', '2025-07-02', '2025-07-03'], dtype='datetime64[ns]', freq='D')
```

Generating Period Ranges: pd.period_range()

- `pd.period_range()` : regular period sequences (e.g., months, quarters)

```
# Monthly periods for Q3 2025
periods = pd.period_range('2025-07', '2025-09', freq='M')
print(periods)
# PeriodIndex(['2025-07', '2025-08', '2025-09'], dtype='period[M]')
```

Generating Timedelta Ranges: pd.timedelta_range()

- `pd.timedelta_range()`: regular duration sequences (e.g., every 2 hours)

```
# Every 2 hours for a day
timedeltas = pd.timedelta_range(start='0h', end='6h', freq='1H30T')
print(timedeltas)
# TimedeltaIndex(['0 days 00:00:00', '0 days 01:30:00', '0 days 03:00:00',
#                 '0 days 04:30:00', '0 days 06:00:00'],
#                dtype='timedelta64[ns]', freq='1H30T')
```

Frequency Codes: Business Essentials

Code	Description	Code	Description
D	Day	B	Business day
W	Week	M	Month end
Q	Quarter end	H	Hour
T	Minute	S	Second

- Suffixes: `MS` (month start), `QS` (quarter start), etc.
- We can combine codes: `2H30T` = 2.5 hours

Resampling vs asfreq in Pandas

- **Resample:** Aggregates data to a new frequency (e.g., monthly sales totals)
- **asfreq:** Changes frequency by selecting or filling values (no aggregation)

```
# Resample: monthly mean from daily data
df.resample('M').mean()
# Result: (example output)
#             sales
# 2025-07-31   215.0
# 2025-08-31   230.0
# 2025-09-30   250.0

# asfreq: select value at month-end, or fill missing
df.asfreq('M', method='ffill')
# Result: (example output)
#             sales
# 2025-07-31   300
# 2025-08-31   300
# 2025-09-30   300
```

Example: Resample vs asfreq (1)

Suppose you have daily sales data for July 2025. We can use `resample` or `asfreq` to extract one aggregated value for the month.

```
import pandas as pd
# Create daily sales data for July 2025
index = pd.date_range('2025-07-01', '2025-07-31', freq='D')
sales = pd.Series([100 + i for i in range(31)], index=index)

# Resample: get the mean sales for the month
monthly_mean = sales.resample('M').mean()
print('Resample (mean):')
print(monthly_mean)
# Result:
# 2025-07-31    115.0
# Freq: M, dtype: float64
```

- `resample('M').mean()` computes the average sales for the month.
- similar to `groupby` but for time series data.

Example: Resample vs asfreq (2)

Now let's see how `asfreq` selects the value at the end of the month:

```
# asfreq: get the sales value on the last day of the month
month_end_value = sales.asfreq('M')
print('asfreq (month-end value):')
print(month_end_value)
# Result:
# 2025-07-31    130
# Freq: M, dtype: int64
```

- `asfreq('M')` selects the sales value on the last day of the month (2025-07-31).

Shifting Time Series Data

- **shift():** Move data forward or backward in time
- Useful for creating lagged features (e.g., yesterday's sales)
- The sign of the argument controls direction:
  - Positive values bring past data into the present (lag)
  - Negative values bring future data into the present (lead)

```
# Shift values by 1 period (e.g., previous day's value)
df['value_lag1'] = df['value'].shift(1)  # lag: past into present

# Shift by -1 period (e.g., tomorrow's value)
df['value_lead1'] = df['value'].shift(-1)  # lead: future into present
```

Rolling Windows in Pandas

- **rolling():** Calculate moving averages or other stats over a window
- Essential for smoothing noisy business data, identifying trends, and analyzing volatility
- Common in sales, finance, web analytics, and operations

Rolling Windows: Business Example

Suppose you have daily sales data and want to understand the underlying trend, removing short-term fluctuations. A rolling mean helps reveal the true business pattern.

```
import pandas as pd
# Simulate daily sales for 2025
index = pd.date_range('2025-01-01', '2025-12-31', freq='D')
sales = pd.Series(100 + (pd.np.random.randn(len(index)).cumsum()), index=index)

# 30-day rolling mean: smooths out daily ups and downs
df = pd.DataFrame({'sales': sales})
df['rolling_mean_30d'] = df['sales'].rolling(30).mean()

# Now you can plot both to compare raw vs. smoothed sales
```

- Rolling windows are widely used for moving averages (sales, stock prices), rolling sums (monthly totals), and rolling standard deviations (volatility).

The `dt` Accessor: Extracting Date Parts

- We can't apply date methods directly to Series, but pandas provides the `dt` accessor, so that we can extract useful date parts like year, month, day, and day of the week from each element in a Series of timestamps.

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day_of_week'] = df['date'].dt.day_of_week
...
```

Summary: Dates & Times for Analytics

✅ Parse and clean real-world date/time data with Python and pandas
✅ Use Timestamps, Periods, and Timedeltas for business analysis
✅ Generate, index, and slice time-based data for reporting
✅ Aggregate and resample time series for business periods
✅ Create lagged and lead features for forecasting and ML
✅ Apply rolling windows to smooth trends and analyze volatility
✅ Extract year, month, weekday, and more for features and reports

Mastering these tools unlocks insights from time-based business data.